

```
struct channel
{
    // 文件描述符(监听、通信)、对应的事件
    int fd;
    int events;
    // 读写回调函数 参数
    handleFunc readCallback;
    handleFunc writeCallback;
    handleFunc destroyCallback; // 释放“所有”资源
    void* arg;
};
```

```
struct dispatcher
{
    // 初始化 IO多路复用 用到的数据块dispatcherData, 返回值是泛型, 兼容三种
    void* (*init)();

    // 添加, 将待检测的fd (封装成了channel类型) 添加到poll, epoll, select上
    // 第二个参数, 取出当前dispatcher用到的数据.epollEvent, fdset, pollfd
    int (*add)(struct channel* channel, struct eventLoop* evLoop);

    // 删除
    int (*remove)(struct channel* channel, struct eventLoop* evLoop);

    // 修改
    int (*modify)(struct channel* channel, struct eventLoop* evLoop);

    // 事件检测
    int (*dispatch)(struct eventLoop* evLoop, int timeout);

    // 清除数据, 关闭fd, 释放内存
    int (*clear)(struct eventLoop* evLoop);
};
```

```
struct dispatcher
pollDispatcher
```

```
struct dispatcher
epollDispatcher
```

```
struct dispatcher
selectDispatcher
```

```
struct channelElement
{
    int type;
    struct channel* channel;
    struct channelElement* next;
};
```

```
struct channelMap
{
    // 类似链表, 但是语义的指针数组并不受固定大小
    // 使用二级指针, 实质: 一个二级指针指向的一个指针数组, 首先是一个数组, 然后数组的每个元素又是一个 struct channel 类型的指针 (struct channel*), 是一个结构体的地址
    // 即: struct channel* list[]
    struct channel** list;
    int size; // 记录指针指向的数组的元素总个数
};
```

```
struct eventLoop
{
    // 保存一个dispatcher实例, 及其数据
    struct dispatcher* dispatcher;
    void* dispatcherData; // 包含epoll回调节点, epoll_event数组

    bool isQuit;
    // 任务队列
    struct channelElement* head;
    struct channelElement* tail;

    // map
    struct channelMap* channelMap;

    // 线程id, 线程名, 互斥锁 (保护任务队列), 条件变量
    pthread_t threadId;
    char threadName[32];
    pthread_mutex_t mutex;

    // 用于与本地通信的fd, 通过socketpair初始化, 一个发一个收, 收的那个用来
    // 主动帮助解除阻塞
    int socketPair[2];
};
```

```
struct workThread
{
    pthread_t threadId;
    char name[24];
    pthread_mutex_t mutex;
    pthread_cond_t cond;

    // 反应堆模型
    struct eventLoop* evLoop;
};
```

```
struct threadPool
{
    // 主线程的反应堆模型, 只做备份用, 处理连接建立, 当没有子
    // 线程的时候, 才处理和客户端的连接
    struct eventLoop* mainLoop;
    bool isStart;
    int threadNum;
    // 子线程数组, 采用指针方式
    struct workThread* workThreads;
    int index;
};
```

```
struct listener
{
    // 用于监听的文件描述符和端口
    int fd;
    unsigned short port;
};
```

```
struct tcpServer
{
    int threadNum;
    struct eventLoop* mainLoop;
    struct listener* listener;
    struct threadPool* threadPool;
};
```

```
enum httpStatusCode
{
    Unknown,
    OK = 200,
    MovedPermanently = 301,
    MovedTemporarily = 302,
    BadRequest = 400,
    NotFound = 404
};
```

```
struct responseHead
{
    char key[32];
    char val[128];
};
```

```
struct httpResponse
{
    // 状态行: 状态码, 状态描述, 版本
    enum httpStatusCode statusCode;
    char statusMsg[128];
    // 响应头, 键值对
    struct responseHead* head;
    int headNum;
    struct responseBody* body;
    char fileName[128];
};
```

```
typedef void (*responseBody)
(const char* fileName, struct
buffer* sendBuf, int socket);
```

```
struct buffer
{
    // 指向实际存储数据的内存指针
    char* data;
    // buffer 内存的总字节数
    int capacity;
    // 读写位置
    int readPos;
    int writePos;
};
```

```
struct tcpConnection
{
    struct eventLoop* evLoop;
    struct channel* channel;
    struct buffer* RBuf;
    struct buffer* WBuf;
    char name[32];

    // http协议
    struct httpRequest* request;
    struct httpResponse* response;
};
```

```
enum httpRequestState
{
    ParseReqLine,
    ParseReqHead,
    ParseReqBody,
    ParseReqDone
};
```

```
struct httpRequest
{
    // 请求行
    char* method;
    char* uri;
    char* version;

    // 请求头的键值对数组
    struct requestHead* reqHead;
    int reqHeadNum;

    enum httpRequestState curState;
};
```

```
struct requestHead
{
    char* key;
    char* value;
};
```